

Primera parte

Proyecto: **Jardinería.**

lenguaje: **Python.**

Framework: **Django.**

Editor: **VS code.**

1 Procedimiento para crear carpeta del Proyecto: **UIII_Jardineria_0376**

2 procedimiento para abrir vs code sobre la carpeta

UIII_Jardineria_0376

3 procedimiento para abrir terminal en vs code

4 Procedimiento para crear carpeta entorno virtual “.venv” desde terminal de vs code

5 Procedimiento para activar el entorno virtual.

6 procedimiento para activar intérprete de python.

7 Procedimiento para instalar Django

8 procedimiento para crear proyecto **backend_Jardineria** sin duplicar carpeta.

9 procedimiento para ejecutar servidor **en el puerto 8036**

10 procedimiento para copiar y pegar el link en el navegador.

11 procedimiento para crear aplicacion **app_Jardineria**

12 Aqui el modelo **models.py**

==--==--==--

```
from django.db import models
```

```
# =====
```

```
# MODELO: CLIENTE
```

```
# =====
```

```
Class Cliente(models.Model):
```

```
    nombre = models.CharField(max_length=100)
```

```
    apellido = models.CharField(max_length=100)
```

```
    telefono = models.CharField(max_length=15, unique=True)
```

```
    email = models.EmailField(unique=True)
```

```
    direccion = models.CharField(max_length=100)
```

```
    ciudad = models.CharField(max_length=100)
```

```
    codigo_postal = models.CharField(max_length=10)
```

```
def __str__(self):
```

```
    return f'{self.nombre} {self.apellido}'
```

```
# =====
```

```
# MODELO: EMPLEADO
```

```
# =====
```

```
class Empleado(models.Model):
```

```
    nombre = models.CharField(max_length=100)
```

```
    apellido = models.CharField(max_length=100)
```

```
    puesto = models.CharField(max_length=100)
```

```
    telefono = models.CharField(max_length=15, unique=True)
```

```
    email = models.EmailField(unique=True)
```

```

    fecha_contratacion = models.DateField()
    salario = models.DecimalField(max_digits=10, decimal_places=2)

    def __str__(self):
        return f"{self.nombre} {self.apellido} ({self.puesto})"

# =====
# MODELO: SERVICIO
# =====
class Servicio(models.Model):
    nombre_servicio = models.CharField(max_length=100)
    descripcion = models.TextField()
    precio = models.DecimalField(max_digits=8, decimal_places=2)
    duracion_estimada_horas = models.DecimalField(max_digits=4,
decimal_places=2)
    fecha_creacion = models.DateTimeField(auto_now_add=True)
    activo = models.BooleanField(default=True)
    tipo_servicio = models.CharField(max_length=50, help_text="Ej: Poda,
Riego, Diseño, Fumigación")

    def __str__(self):
        return self.nombre_servicio
=====

# Modelo ContratoServicio (Relación de 1 a muchos: Un cliente puede tener
muchos contratos de servicio)
# (Relación de muchos a muchos: Un contrato de servicio puede ser realizado
por varios empleados, y un empleado puede participar en varios contratos)
class ContratoServicio(models.Model):
    cliente = models.ForeignKey(Cliente, on_delete=models.CASCADE,
related_name='contratos')
    servicio = models.ForeignKey(Servicio, on_delete=models.CASCADE,
related_name='contratos')
    empleados = models.ManyToManyField(Empleado,
related_name='servicios_asignados')
    fecha_contrato = models.DateField(auto_now_add=True)
    fecha_programada = models.DateField()
    estado = models.CharField(
        max_length=20,
        choices=[
            ('PENDIENTE', 'Pendiente'),
            ('EN_PROGRESO', 'En Progreso'),
            ('COMPLETADO', 'Completado'),
            ('CANCELADO', 'Cancelado'),
        ],
        default='PENDIENTE'
    )
    observaciones = models.TextField(blank=True, null=True)
    costo_total = models.DecimalField(max_digits=10, decimal_places=2,
blank=True, null=True) # Se puede calcular

    def __str__(self):
        return f"Contrato {self.id} - {self.cliente} - {self.servicio}"

```

12.5 Procedimiento para realizar las migraciones(makemigrations y migrate.

13 primero trabajamos con el **MODELO: CLIENTES**

14 En view de app_Jardineria crear las funciones con sus códigos correspondientes (inicio_jardineria, agregar_cliente, actualizar_cliente, realizar_actualizacion_cliente, borrar_cliente)

15 Crear la carpeta "templates" dentro de "**app_Jardineria**".

16 En la carpeta templates crear los archivos html (**base.html, header.html, navbar.html, footer.html, inicio.html**).

17 En el archivo base.html **agregar bootstrap** para css y js.

18 En el archivo navbar.html incluir las opciones ("Sistema de Administración Jardineria", "Inicio", "cliente", en submenu de clientes(Agregar Cliente, ver cliente, actualizar cliente, borrar cliente), "Empleado" en submenu de Empleado(Agregar empleado, ver empleado, actualizar empleado, borrar empleado) "Servicio" en submenu de Servicio (Agregar servicio, ver servicio, actualizar servicio, borrar servicio), incluir iconos a las opciones principales, no en los submenu.

19 En el archivo footer.html incluir derechos de autor, fecha del sistema y "Creado por Tecnico Lizet Silva, Cbtis 128" y mantenerla fija al final de la página.

20 En el archivo inicio.html se usa para colocar información del sistema más una imagen tomada desde la red sobre cinopolis.

21 Crear la subcarpeta carpeta categoria dentro de app_Jardineria\templates.

22 crear los archivos html con su código correspondientes de (agregar_cliente.html, ver_cliente.html mostrar en tabla con los botones ver, editar y borrar, actualizar_cliente.html, borrar_cliente.html) dentro de app_Jardineria\templates\cliente.

23 No utilizar forms.py.

24 procedimiento para crear el archivo urls.py en app_Jardineria con el código correspondiente para acceder a las funciones de views.py para operaciones de crud en cliente.

25 procedimiento para agregar app_Jardineria en settings.py de backend_Jardineria

26 realizar las configuraciones correspondiente a urls.py de backend_Jardineria para enlazar con app_Jardineria

27 procedimiento para registrar los modelos en admin.py y volver a realizar las migraciones.

28 por lo pronto solo trabajar con "cliente" dejar pendiente # **MODELO: EMPLEADO** y # **MODELO: SERVICIO**

29 Utilizar colores suaves, atractivos y modernos, el código de las páginas web sencillas.

30 No validar entrada de datos.

31 Al inicio crear la estructura completa de carpetas y archivos.

32 proyecto totalmente funcional.

33 finalmente ejecutar servidor en el puerto puerto 8036.